

Task 1

```
package Day26;
public class ChessGame { 2 usages
    private DifficultyLevel difficultyLevel; 3 usages

    // Setter for strategy
    public void setDifficultyLevel(DifficultyLevel difficultyLevel) { 3 usages
        this.difficultyLevel = difficultyLevel;
    }

    // Executes the chosen strategy
    public void startChessGame() { 3 usages
        if (difficultyLevel != null) {
            difficultyLevel.play(); // Delegates to current strategy
        } else {
            System.out.println("Please set a difficulty level first.");
        }
    }
}
```

```
package Day26;
public class DifficultLevel implements DifficultyLevel { 1 usage
    @Override 1 usage
    public void play() {
        System.out.println("This is a Difficult level game");
    }
}
```

```
package Day26;
public interface DifficultyLevel { 5 usages 3 implementations
    void play(); 1 usage 3 implementations
}
```

```
package Day26;

public class EasyLevel implements DifficultyLevel {
    @Override
    public void play() {
        System.out.println("This is an Easy level game");
    }
}
```

```
package Day26;

public class MediumLevel implements DifficultyLevel {
    @Override
    public void play() {
        System.out.println("This is a Medium level game");
    }
}
```

```
package Day26;

public class ChessGameMain {
    public static void main(String[] args) {
        System.out.println("Strategy Design Pattern = Behavioral Design");

        ChessGame game = new ChessGame();

        game.setDifficultyLevel(new EasyLevel());
        game.startChessGame();

        game.setDifficultyLevel(new MediumLevel());
        game.startChessGame();

        game.setDifficultyLevel(new DifficultLevel());
        game.startChessGame();
    }
}
```

↑ "D:\Program Files\Java\jdk-24\bin
 ↓ Strategy Design Pattern = Behavior
 This is an Easy level game
 This is a Medium level game
 This is a Difficult level game
 Process finished with exit code 1

Task 2 - What do you know about Design Work Flow...

A **design workflow** is the structured process or sequence of steps that a designer (or design team) follows to take an idea from the concept stage to a finished and implemented design.

Planning → Define features, schedule, tech stack.

Execution → Developers code, testers test.

Monitoring → Project manager tracks progress.

Closure → Final deployment & retrospective.

Task 3: What do you know Persistent objects?

A **persistent object** is an object whose state (data) outlives the program that created it.

Normally, when you create objects in memory (RAM), they disappear once the program ends. But persistent objects are stored permanently (like in a database, file, or storage system) so that:

- They can be retrieved later, even after the application restarts.
- Their state is maintained between sessions.

Task 4:

Which of the following components is not typically part of the Command pattern?

- a) Invoker
- b) Receiver
- c) Abstract Factory
- d) Command (interface/abstract class)

Task 5:

What role does the Invoker play in the Command pattern?

- a) It knows how to perform the operations associated with a request.
- b) It encapsulates the request as an object.
- c) It asks the command to carry out the request.
- d) It defines the interface for executing an operation.

Task 6

A key benefit of using the Command pattern is its ability to support:

- a) Lazy initialization
- b) Undo/Redo functionality

- c) Singleton instance creation
- d) Compile-time polymorphism

Task 7:

In the Strategy pattern, what role does the "Context" play?

- A. It defines the interface for the algorithms.
- B. It implements a specific algorithm.
- C. It maintains a reference to a Strategy object and delegates the task to it.
- D. It creates the Concrete Strategy objects.

Task 8:

In which of the following mechanisms, types of all variables and expressions are fixed at compilation time.

- a) Strong Typing
- b) Weak Typing
- c) Static Binding/ early binding
- d) Dynamic Binding/ late binding

Task 9:

In which pattern does a class represent the functionality of another class, providing a simplified interface to a complex subsystem?

- a) Decorator Pattern
- b) Facade Pattern
- c) Proxy Pattern
- d) Composite Pattern

Task 10:

Which of the following statements about Persistence is correct?

- a) It is the enforcement of the class of an object, such that objects of different types may not be interchanged, or at the most they may be interchanged only in very restricted ways.
- b) It is the property of an object through which its existence transcends time and/or space.
- c) It is the property that distinguishes an active object from one that is not active.
- d) All of the mentioned

Task 11:

What is that concept in type theory in which a single name may denote objects of many different classes that are related by some common super class referred to _____

- a) Monomorphism
- b) Type Checking
- c) Polymorphism
- d) Generalization

Task 12

Which of the following patterns is used to create a single instance of a class and provide a global point of access to it?

- a) Factory Pattern
- b) Singleton Pattern
- c) Builder Pattern
- d) Prototype Pattern

Task 13

The Adapter pattern is a type of _____ pattern.

- a) Creational
- b) Structural
- c) Behavioral
- d) Concurrency

Task 14

Which design pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically?

- a) Strategy Pattern
- b) Command Pattern
- c) Observer Pattern
- d) Mediator Pattern

Task 15

The Model-View-Controller (MVC) is an example of a _____ pattern.

- a) Creational
- b) Structural

c) Behavioral

d) Architectural

Task 16

```
package Day26;
import java.util.ArrayList;
import java.util.List;
public class DManager {
    private static DManager instance;
    private List<String> itemList;
    private DManager() {
        itemList = new ArrayList<>();
    }
    public static synchronized DManager getInstance() {
        if (instance == null) {
            instance = new DManager();
        }
        return instance;
    }
    public static DManager createInstance() {
        if (instance != null) {
            throw new IllegalStateException("Instance already exists!")
        }
        instance = new DManager();
        return instance;
    }
    public synchronized void addItem(String item) {
        itemList.add(item);
    }
}
```

```

        itemList.add(item);
        System.out.println("Item added: " + item);
    }

    public synchronized boolean removeItem(String item) { 1 usage
        boolean removed = itemList.remove(item);
        if (removed) {
            System.out.println("Item removed: " + item);
        }
        return removed;
    }

    public synchronized List<String> retrieveList() { 2 usages
        return new ArrayList<>(itemList); // Return a copy to maintain
    }
}

// Main class should be outside DManager
class Main {
    public static void main(String[] args) {
        DManager manager = DManager.getInstance();
        manager.addItem("Item 1");
        manager.addItem("Item 2");
        manager.addItem("Item 3");
        // Retrieve and print list
        System.out.println("Current list: " + manager.retrieveList());
    }
}

```

